ChatB2B – Rapport final

Loïc Boulanger · Charlotte Bizel — 5IDEV, Projet 1 — version du 28 octobre 2026

ChatB2B est une messagerie temps réel pour équipes, en production sur Render et Neon, développée par Loïc Boulanger (backend) et Charlotte Bizel (frontend) : 97 tests automatisés et 18 Pull Requests ont accompagné sa construction.

Sommaire

1. Contexte et objectifs 1
2. Équipe et méthode de travail 2
3. Fonctionnalités réalisées 2
4. Acteurs du système et permissions 5
5. Exigences : MVP et non fonctionnelles 6
6. Choix de la stack technique et justification 7
7. Architecture et déploiement 8
8. Modélisation UML 9
9. Authentification et session 10
10. Messagerie, groupes et notifications 11
11. Pièces jointes 14
12. Réactions et emojis 15
13. Suppression de compte et RGPD 16
14. Qualité logicielle et CI/CD 17
15. Sécurité 19
16. Déploiement et environnements 19
17. Sauvegarde et restauration 20
18. Difficultés rencontrées et leçons 22
19. Limites connues et perspectives 22
20. Conclusion 23
21. Annexes 23

1. Contexte et objectifs

ChatB2B est un projet académique (25 % de la note, à rendre le 2 novembre 2026) : concevoir, tester et déployer en production une messagerie d’équipe en temps réel, avec un dépôt GitHub public et un rapport.

Le cahier des charges fixait un socle commun, auquel l’équipe a ajouté plusieurs fonctionnalités :

- **Socle demandé** : inscription et connexion sécurisées, conversations privées (1:1) et de groupe, messagerie en temps réel, notifications des nouveaux messages, permissions d’accès, interface responsive, tests automatisés et déploiement continu.
- **Ajouts de l’équipe** : pièces jointes, réactions et sélecteur d’emojis, session mémorisée (« Se souvenir de moi »), profil utilisateur et suppression de compte conforme au RGPD, avec anonymisation.

Le projet est en ligne et utilisable : le site est servi par Render, les données sont stockées dans une base PostgreSQL hébergée par Neon, et chaque modification fusionnée dans la branche principale est testée puis déployée automatiquement.

2. Équipe et méthode de travail

Le travail est réparti en deux rôles complémentaires, avec une relecture croisée de chaque modification.

Membre	Rôle	Responsabilités principales
Loïc Boulanger	Backend	API Express, Socket.io, base de données Prisma/PostgreSQL, migrations, tests, déploiement, fonctionnalités ajoutées (pièces jointes, réactions, anonymisation, session)
Charlotte Bizel	Frontend	Interface React (connexion, liste de conversations, groupes, avatars), relecture des Pull Requests

L'équipe applique le **GitHub Flow** :

1. Une branche par fonctionnalité, nommée `feature/...` (par exemple `feature/reactions-backend`).
2. Une Pull Request vers `main`, avec un message de commit explicite (`feat: ...`).
3. Des vérifications automatiques (ESLint, Prettier, tests, build) qui doivent être vertes.
4. Une review par Charlotte avant la fusion.
5. La fusion dans `main`, qui déclenche le déploiement automatique sur Render.

Au 13 octobre 2026, 18 Pull Requests ont été fusionnées sur le dépôt `Qoquan/chatB2B` (voir la figure au chapitre 14).

3. Fonctionnalités réalisées

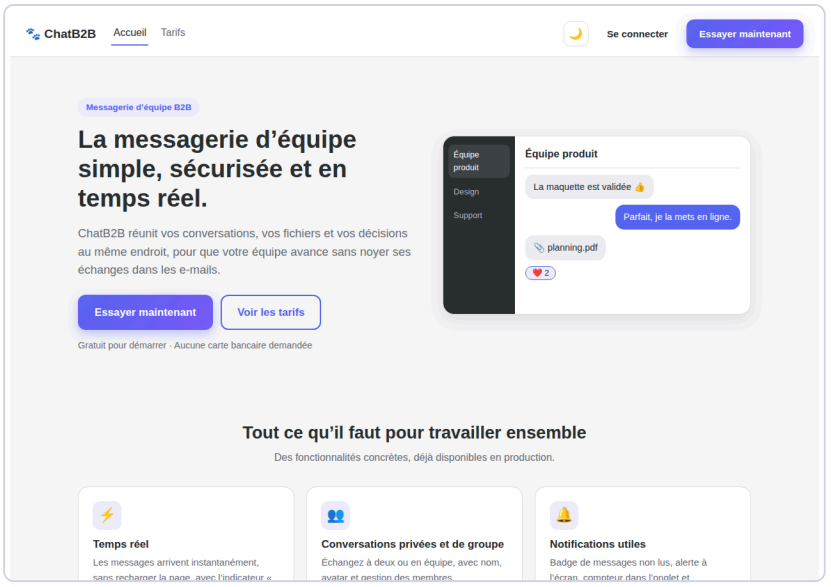
Toutes les fonctionnalités ci-dessous sont en production. Seule la pagination de l'historique n'est pas réalisée.

Fonctionnalité	Description	État
Inscription et connexion	Jeton JWT, mots de passe hachés (bcrypt), validation des champs	Livré
Session mémorisée	Case « Se souvenir de moi pendant 8 jours », session rouverte au rechargement	Livré

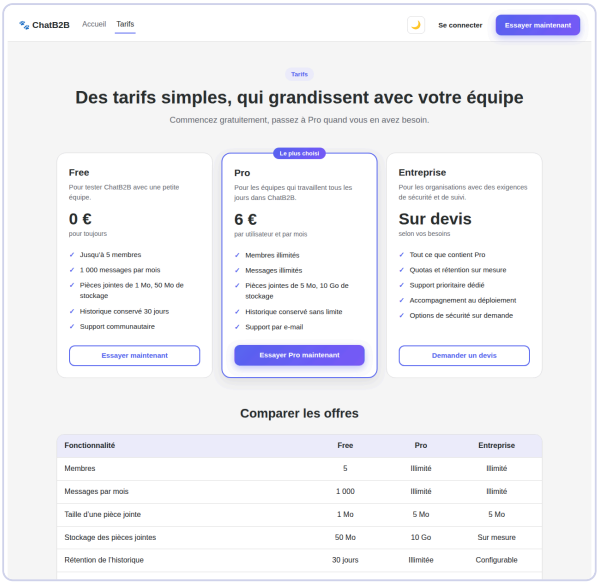
Fonctionnalité	Description	État
Conversations privées et de groupe	Création, renommage, ajout de membres, suppression, avatar de groupe	Livré
Messagerie en temps réel	Envoi et réception instantanés (Socket.io), indicateur « en train d'écrire »	Livré
Messages non lus et notifications	Badge, message flash, titre de l'onglet « (n) ChatB2B », notification du navigateur	Livré
Pièces jointes	Images, PDF, documents Office, texte ; 5 Mo maximum ; contrôle du contenu réel	Livré
Réactions et emojis	6 réactions par message, sélecteur d'emojis, emojis seuls affichés en grand	Livré
Profil	Pseudo, avatar, changement de mot de passe	Livré
Suppression de compte	Anonymisation conforme au RGPD, messages conservés sous « Utilisateur supprimé »	Livré
Tests et déploiement continu	97 tests, GitHub Actions, déploiement automatique sur Render	Livré
Pagination de l'historique	Chargement des messages par pages	Non réalisé
Landing page et page tarifs	Accueil public et tarifs Free / Pro / Entreprise avec quotas, mode sombre	Livré

3.1 Page d'accueil et tarifs

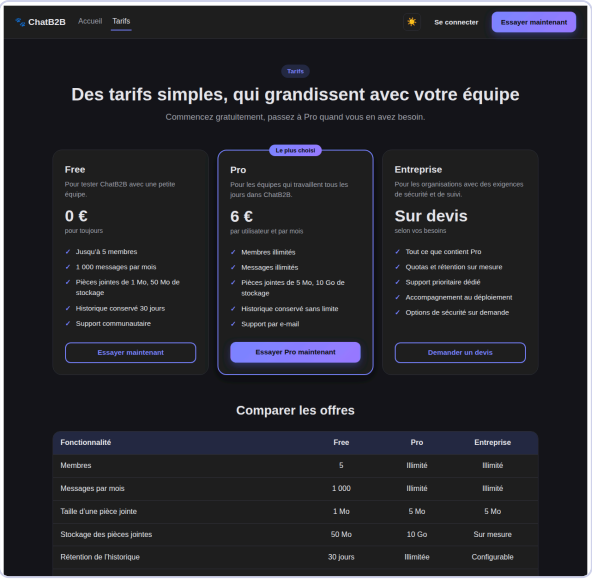
Comme pour un produit SaaS B2B, ChatB2B dispose d'une vitrine publique accessible sans compte. La **page d'accueil** présente la promesse du produit, six fonctionnalités, les trois étapes pour démarrer et un bouton « Essayer maintenant ». La **page de tarifs** propose trois offres : **Free** (0 €), **Pro** (6 € par utilisateur et par mois) et **Entreprise** (sur devis), avec des quotas sur les membres, les messages, les pièces jointes et la rétention de l'historique, un tableau comparatif et une foire aux questions.



Page d'accueil



Page de tarifs (mode clair)



Page de tarifs (mode sombre)

Les pages publiques proposent un mode clair et un mode sombre, mémorisé dans le navigateur, et s'adaptent aux petits écrans. Elles utilisent un routeur par fragment (adresses du type `/#/pricing`), qui ne demande aucune configuration sur l'hébergement statique. Les prix et les limites des offres sont fictifs : aucun paiement n'est proposé et les quotas ne sont pas appliqués par l'application.

4. Acteurs du système et permissions

Quatre profils d'utilisateurs et un acteur technique (le serveur Socket.io) interagissent avec ChatB2B. Les droits sont contrôlés côté serveur et non seulement dans l'interface : chaque route vérifie le jeton, puis l'appartenance à la conversation.

Acteur	Description	Peut	Ne peut pas
Visiteur	Personne non connectée	Consulter l'accueil et les tarifs, s'inscrire, se connecter	Accéder à une route protégée (réponse 401) ou à une conversation
Utilisateur connecté	Membre actif	Gérer son profil, créer des conversations privées et des groupes, lire et envoyer des messages, pièces jointes et réactions dans ses conversations, renommer un groupe et y ajouter des membres, supprimer son compte	Lire ou écrire dans une conversation dont il n'est pas membre (réponse 403), modifier une conversation privée
Créateur d'un groupe	Membre qui a créé le groupe	Tout ce que fait un membre, plus la suppression du groupe	Perdre ce rôle, sauf suppression de son compte (le rôle passe au plus ancien membre actif)

Acteur	Description	Peut	Ne peut pas
Compte supprimé	Ancien utilisateur anonymisé	Rien : ses anciens messages restent affichés sous « Utilisateur supprimé »	Se connecter

Le serveur Socket.io est l'acteur technique du temps réel : il authentifie chaque connexion, place l'utilisateur dans les salles de ses conversations et diffuse les messages, les réactions et les changements de conversation.

Principales interactions : le visiteur s'inscrit ou se connecte et reçoit un jeton ; l'utilisateur ouvre une conversation, le serveur vérifie qu'il en est membre, enregistre le message puis le diffuse aux autres membres connectés ; un membre peut réagir ou joindre un fichier, sous les mêmes contrôles.

Action	Qui	Contrôle côté serveur
Lire l'historique d'une conversation	Ses membres	Jeton valide et appartenance à la conversation (403 sinon)
Envoyer un message ou une pièce jointe	Ses membres	Idem ; refus si l'autre compte d'une conversation privée est supprimé
Télécharger une pièce jointe	Ses membres	Route protégée, appartenance vérifiée
Renommer un groupe, changer son avatar	Tout membre du groupe	Groupes uniquement
Ajouter des membres	Tout membre du groupe	Groupes uniquement
Supprimer un groupe	Son créateur	Réponse 403 pour tout autre membre

5. Exigences : MVP et non fonctionnelles

5.1 MVP du cahier des charges

Exigence du MVP	Réponse apportée	État
Conversations privées (1:1) et de groupe	Création, membres, historique des messages	Livré
Temps réel sans rechargement de page	Socket.io, diffusion par salles	Livré
Notifications en temps réel	Badge de non-lus, toast, titre de l'onglet, notification du navigateur	Livré
Authentification et permissions	JWT, contrôle d'appartenance à chaque conversation	Livré

Exigence du MVP	Réponse apportée	État
Interface responsive	Mise en page adaptative (point de rupture à 768 px dans le chat, pages publiques adaptées au mobile)	Livré
Optionnel : stockage de fichiers	Pièces jointes sécurisées, 5 Mo	Livré
Optionnel : recherche plein texte	Non réalisée	Non réalisé
Landing page et pricing	Accueil public et trois offres avec quotas	Livré

5.2 Exigences non fonctionnelles

Exigence	Mise en œuvre
Temps réel	WebSocket via Socket.io : messages, réactions et changements de conversation diffusés immédiatement
Sécurité	Mots de passe hachés, JWT, validation et échappement des entrées, origines CORS en liste blanche, contrôle d'accès, fichiers vérifiés (voir chapitre 15)
Protection des données	Anonymisation du compte à sa suppression (voir chapitre 13)
Gestion des erreurs	Gestionnaire d'erreurs central : message JSON clair, jamais de trace technique envoyée au client ; messages d'erreur explicites dans les formulaires ; écran de chargement pendant la reconnexion
Qualité et maintenabilité	ESLint, Prettier, 97 tests automatisés, migrations de base versionnées, intégration continue
Déploiement	Déploiement automatique à chaque fusion dans la branche principale ; sur l'offre gratuite de Render, le serveur peut se mettre en veille et mettre quelques secondes à répondre au premier chargement
Ergonomie	Interface adaptative, mode sombre sur les pages publiques

6. Choix de la stack technique et justification

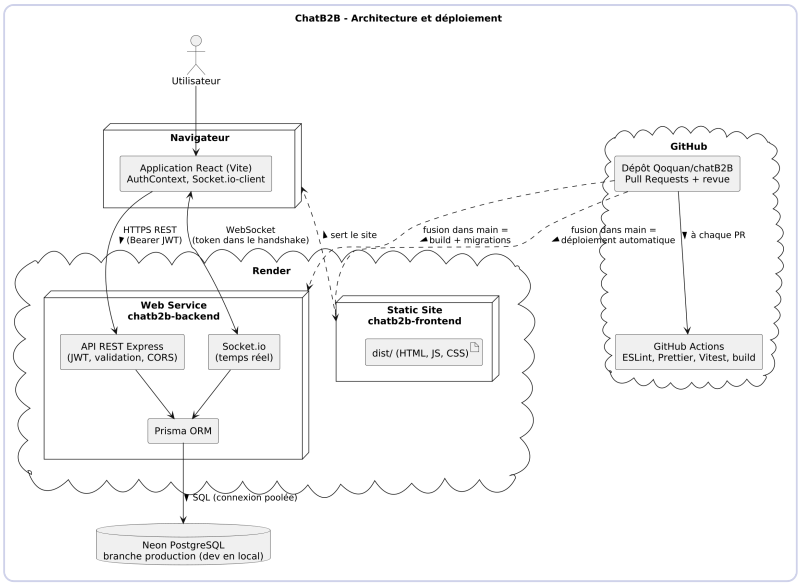
Le choix de l'hébergement a guidé celui de la stack : l'application a besoin de persistance des données et de WebSockets, ce que tous les hébergeurs ne permettent pas.

Couche	Choix	Justification
Hébergement	Render (service web et site statique)	Services Node.js persistants qui acceptent les WebSockets, déploiement automatique depuis GitHub, offre gratuite adaptée à un projet scolaire
Base de données	PostgreSQL chez Neon	Base relationnelle adaptée au modèle (utilisateurs, conversations, membres, messages), branches production et dev séparées, offre gratuite
Backend	JavaScript (Node.js) avec Express 4	Langage interprété imposé, même langage que le frontend, framework léger et très documenté
Temps réel	Socket.io 4	Salles par conversation, reconnexion automatique, repli si le WebSocket est indisponible
Accès aux données	Prisma 5	Requêtes paramétrées, migrations versionnées, schéma lisible
Frontend	React 18 avec Vite	Composants réutilisables, développement rapide, build statique servi gratuitement
Authentification	JWT et bcrypt	Jeton sans état qui fonctionne entre deux domaines, mots de passe hachés
Qualité et CI	Vitest, supertest, ESLint, Prettier, GitHub Actions	Tests de l'API et du temps réel, vérification automatique de chaque Pull Request

7. Architecture et déploiement

ChatB2B suit une architecture client–serveur classique, séparée en deux applications déployées indépendamment. Le **frontend** (React 18 + Vite) est servi sur Render comme site statique. Le **backend** (Node.js / Express 4 + Socket.io 4) est un service web Render. Il expose une API REST pour l'authentification, les utilisateurs, les conversations, les messages, les pièces jointes et les réactions. Il ouvre aussi un canal WebSocket pour le temps réel. Les données sont stockées dans **PostgreSQL hébergé chez Neon**, accessible via **Prisma 5.22**.

Le code vit sur GitHub (`Qoquan/chatB2B`). Chaque fusion dans `main` déclenche un déploiement automatique sur Render, dont la commande de build est `npm install && npx prisma generate && npx prisma migrate deploy` : les migrations de base de données sont donc appliquées à chaque déploiement.



Architecture et déploiement de ChatB2B

Deux choix méritent d'être expliqués. Le jeton JWT est envoyé dans l'en-tête `Authorization` et non dans un cookie, car le frontend et le backend sont sur deux domaines Render différents et les navigateurs bloquent les cookies tiers. Neon fournit deux URL : une URL poolée (`DATABASE_URL`) pour l'application et une URL directe (`DIRECT_URL`) pour les migrations.

8. Modélisation UML

8.1 Diagramme de classes

Le diagramme ci-dessous représente le modèle de données (schéma Prisma). Un **User** est membre de plusieurs **Conversation** via la classe d'association **ConversationMember**. Un utilisateur peut aussi être le créateur d'une conversation de groupe. Une **Conversation** *contient* des **Message** (composition : un message n'existe pas sans sa conversation). Un **Message** *possède* des **Attachment** (pièces jointes, stockées en BYTEA) et des **Reaction** (une par couple message / utilisateur / emoji).

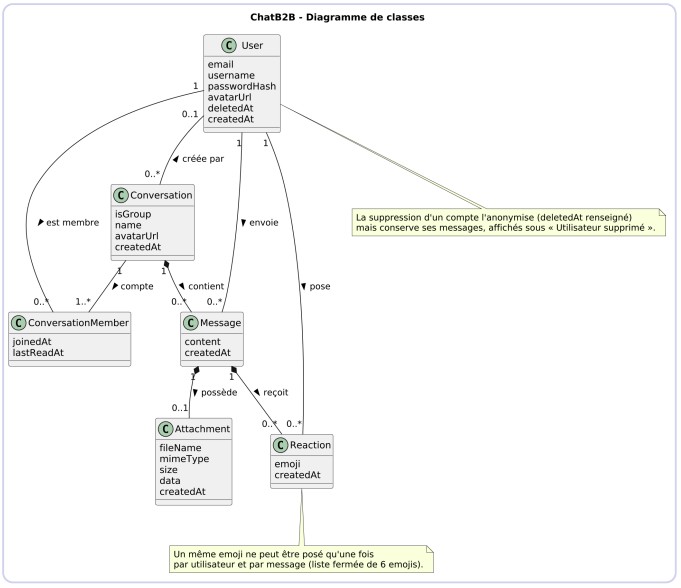


Diagramme de classes

L'anonymisation d'un compte (RGPD) repose sur le champ `deletedAt` de `User` : l'e-mail et le pseudo sont remplacés, le mot de passe est écrasé par une valeur aléatoire, mais les messages restent.

8.2 Diagramme de séquence : envoi d'un message

Le diagramme suivant décrit l'envoi d'un message : le client émet `send_message` via Socket.io, le serveur vérifie le jeton et l'appartenance à la conversation, enregistre le message via Prisma, puis diffuse `new_message` à tous les membres connectés de la salle `conversation:<id>`.

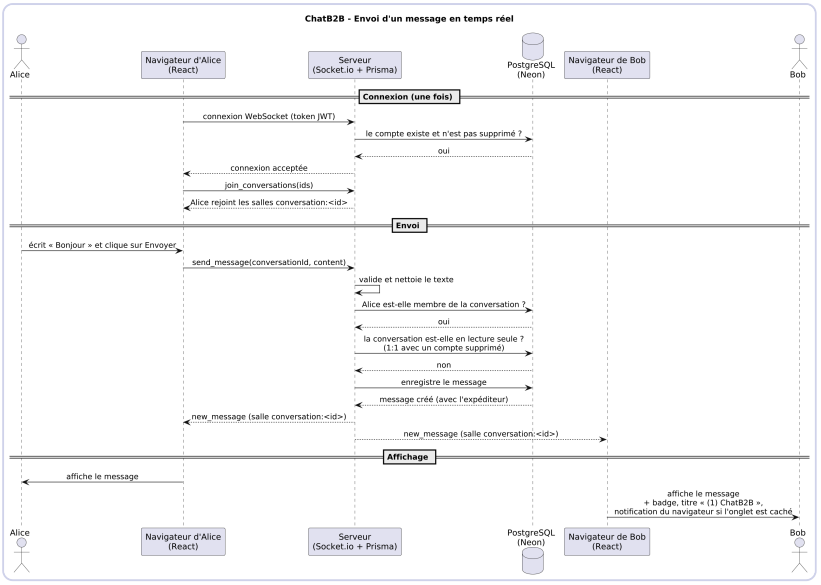
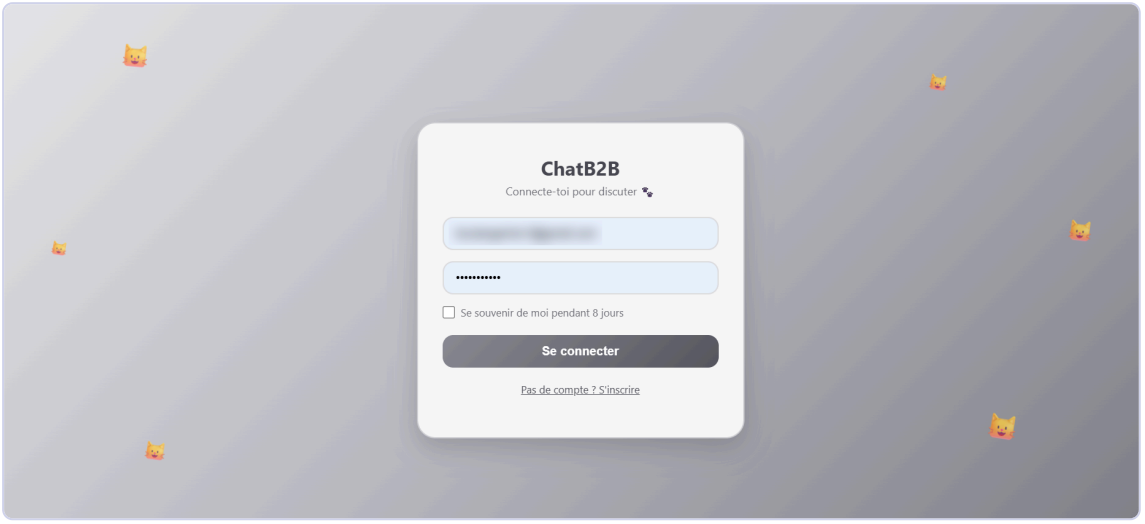


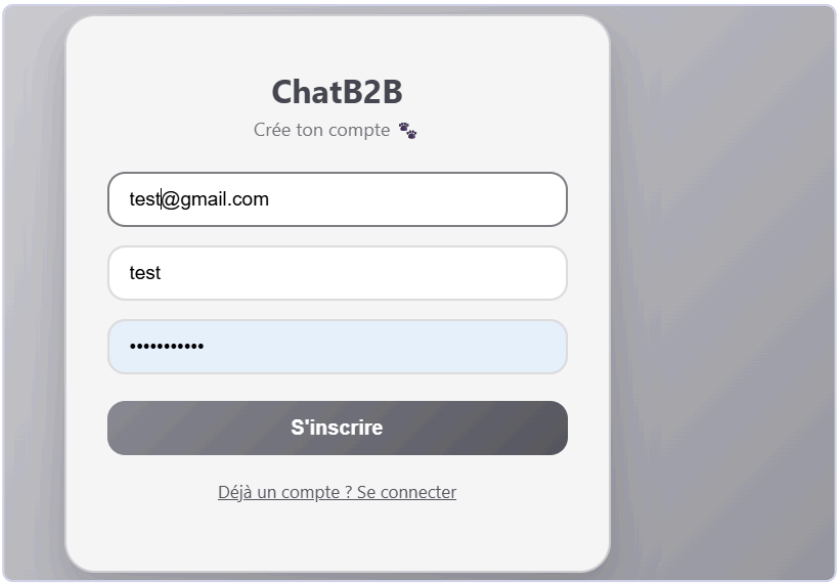
Diagramme de séquence de l'envoi d'un message

9. Authentification et session

L'inscription demande un e-mail, un pseudo et un mot de passe. Le mot de passe est haché avec **bcrypt** avant stockage et n'est jamais conservé en clair. À la connexion, le serveur renvoie un **JWT**.



Page de connexion



Page d'inscription

La session est **persistante** : au chargement de l'application, le frontend relit le jeton stocké et appelle `GET /api/users/me` pour restaurer l'utilisateur. Si le serveur répond 401, le jeton est effacé ; en cas d'erreur réseau ou 5xx, le jeton est conservé. Pendant la vérification, un écran de chargement est affiché.

La case **« Se souvenir de moi pendant 8 jours »** change la durée du jeton (8 jours au lieu de 1 jour) et son stockage : `localStorage` si la case est cochée, `sessionStorage` sinon. Le serveur n'accepte que le booléen strict `true` pour cette option. Les identifiants (mot de passe) ne sont jamais mémorisés : seul un jeton signé l'est. Un cookie n'a pas été retenu, pour la raison de domaines différents expliquée au chapitre 7.

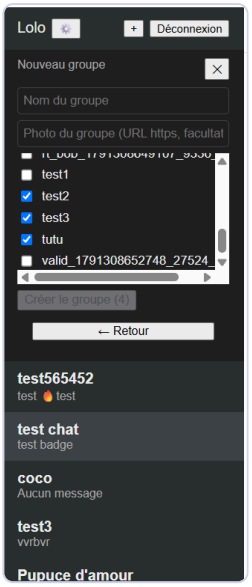
10. Messagerie, groupes et notifications

Les messages sont envoyés et reçus en **temps réel** via Socket.io. Chaque utilisateur rejoint une salle par conversation (`conversation:<id>`) et une salle personnelle (`user:<id>`). Les événements principaux sont `send_message`, `new_message`, `typing` / `user_typing` (indicateur de saisie), `conversation_created`, `conversation_updated` et `conversation_deleted`. Le socket est authentifié par le même JWT que l'API.



Vue principale de l'application

L'application permet les conversations privées (1 à 1) et les **conversations de groupe**, avec nom, avatar et un créateur. Si le créateur supprime son compte, ce rôle passe au membre actif le plus ancien.



Création d'un groupe

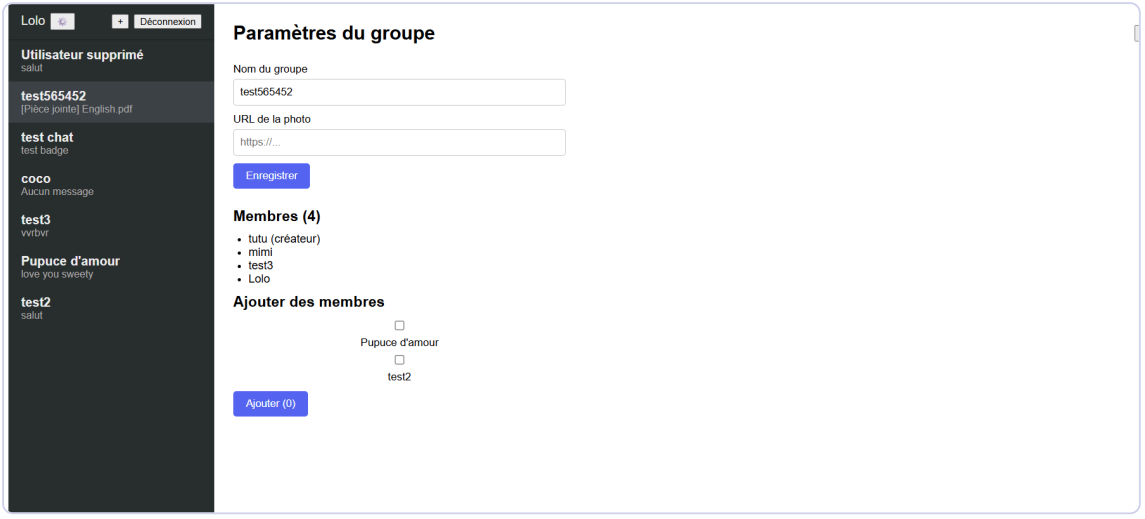
Quatre mécanismes de **notification** préviennent l'utilisateur : un badge de messages non lus par conversation, une notification interne (toast), le titre de l'onglet du navigateur sous la forme « (n) ChatB2B », et une notification native du navigateur lorsque l'onglet est masqué (comportement variable selon le navigateur).



Titre de l'onglet avec compteur



Badge de notification

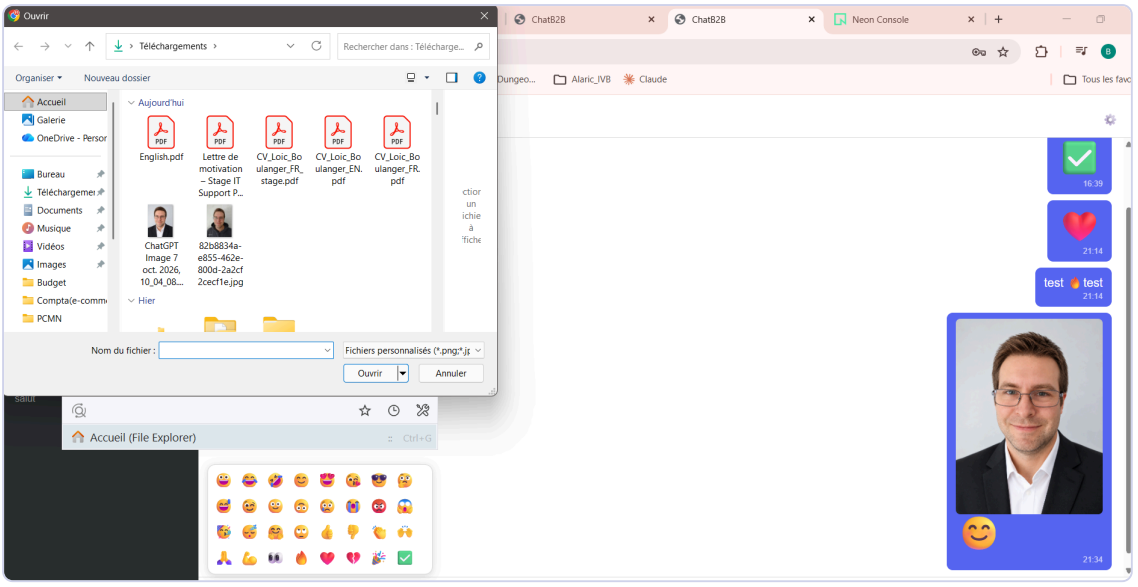


Paramètres d'un groupe : nom, photo, membres et ajout de membres

11. Pièces jointes

Un message peut contenir une pièce jointe de **5 Mo maximum**. Dix types sont acceptés : PNG, JPEG, GIF, WebP, PDF, DOCX, XLSX, PPTX, TXT et CSV. Les fichiers sont stockés directement en base (colonne `BYTEA`) et ne sont pas accessibles publiquement : le téléchargement passe par une route protégée qui exige d'être membre de la conversation.

Chaque fichier subit trois contrôles : le **type MIME** déclaré, l'**extension**, et les **magic bytes** (les premiers octets réels du fichier), ce qui empêche de déguiser un exécutable en image. Les SVG et HTML sont refusés car ils peuvent contenir du code actif. L'appartenance à la conversation est vérifiée avant même de recevoir le fichier. Le téléversement utilise `multer` en mémoire.

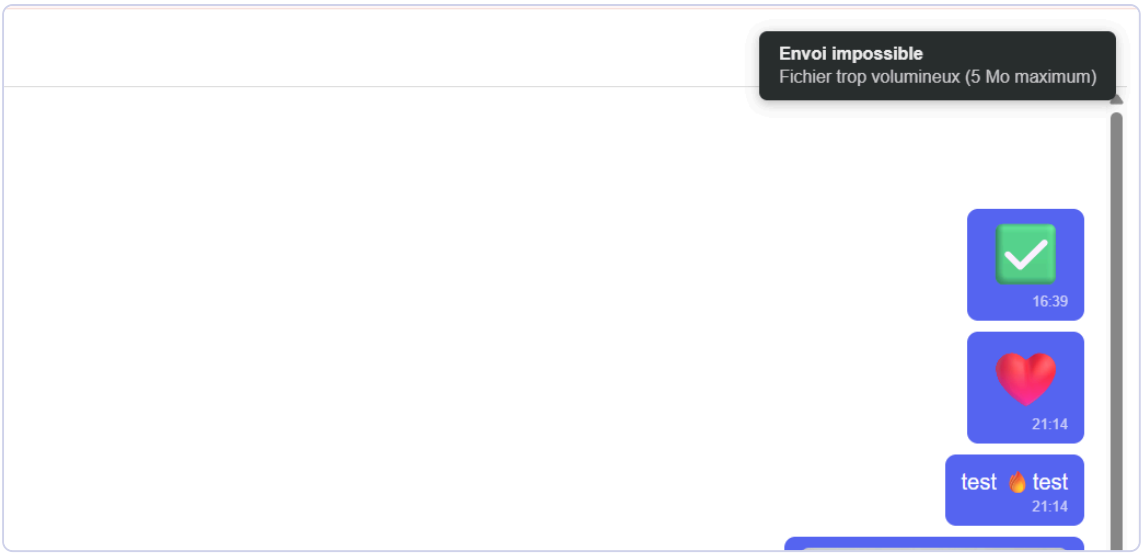


Envoi d'une image



Pièce jointe PDF

Un fichier dépassant la limite est refusé dès l'envoi, avec un message explicite :

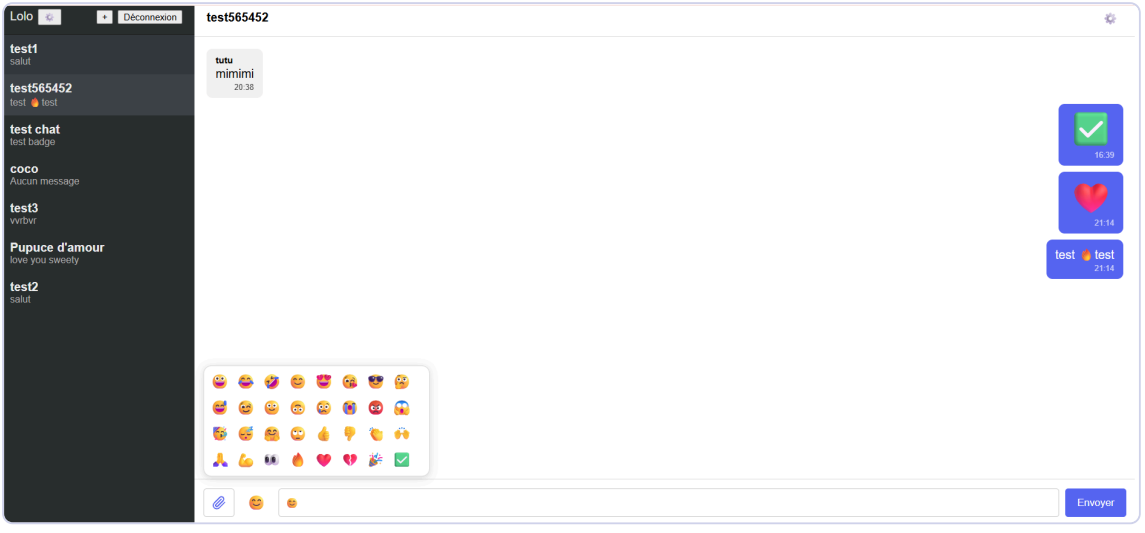


Refus d'un fichier trop volumineux (5 Mo maximum)

12. Réactions et emojis

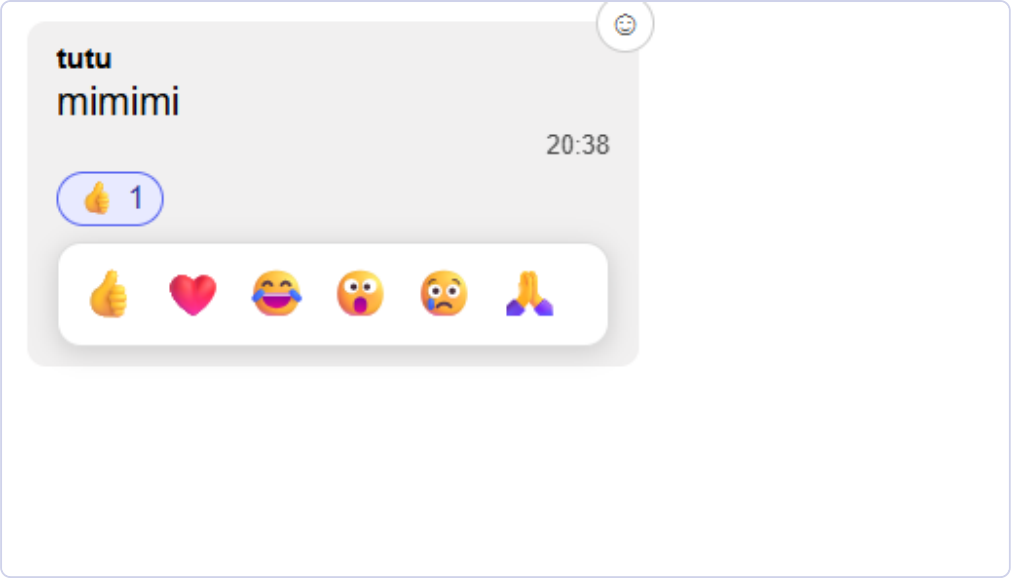
Les utilisateurs peuvent réagir à un message avec l'un des six emojis autorisés : 👍 ❤️ 😂 😮 😢 🙏. Cette liste est une **liste blanche** vérifiée côté serveur. Une contrainte d'unicité (message, utilisateur, emoji) empêche les doublons. Une seule route `POST` fait office de bascule : réagir ajoute la réaction, réagir à nouveau la retire. Le serveur diffuse ensuite l'événement `reaction_updated` pour que tous les participants voient le changement immédiatement.

Côté interface, un sélecteur d'emojis permet aussi d'en insérer dans un message. Quand un message ne contient **que des emojis**, ils sont affichés en grand (par rangées de deux) ; dans un texte, ils gardent la taille normale. Les réactions conservent aussi leur taille normale.



Sélecteur d'emojis et emojis en grand

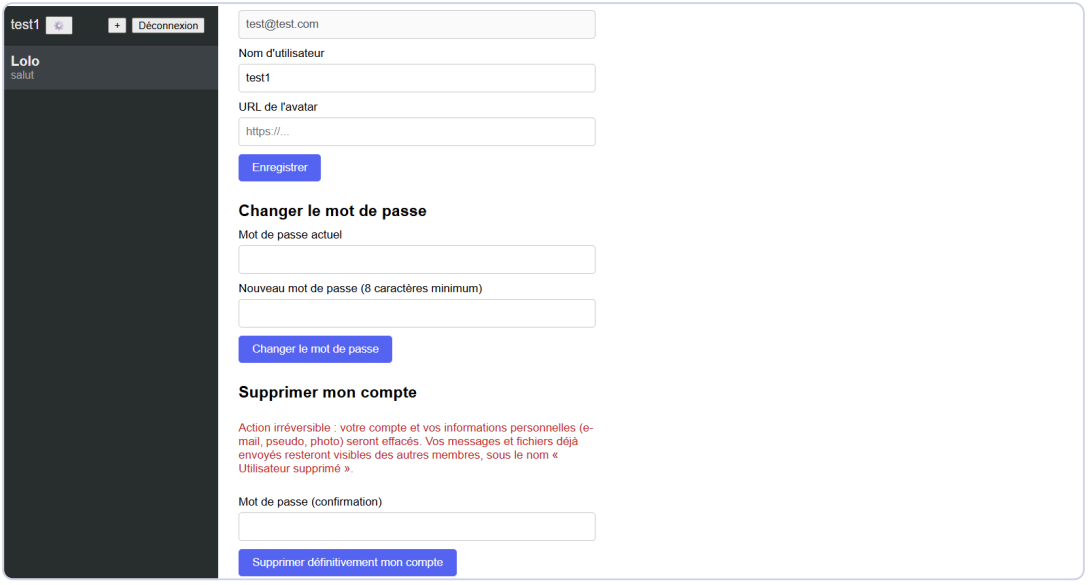
Le menu de réaction propose les six emojis ; les réactions déjà données apparaissent sous le message avec leur compteur.



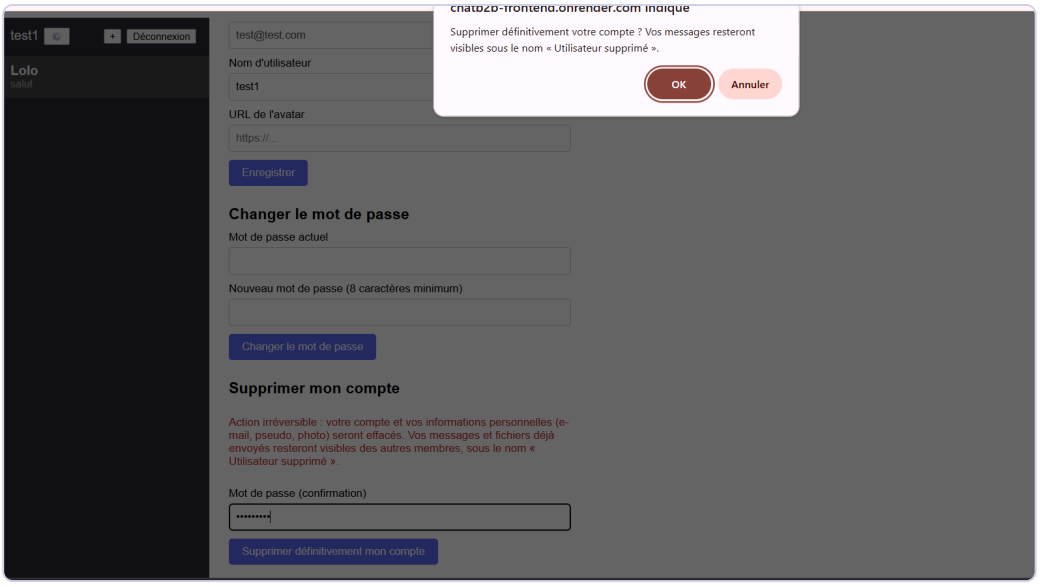
Menu de réaction et pastille de réaction

13. Suppression de compte et RGPD

Chaque utilisateur peut supprimer son compte depuis son profil, après une confirmation explicite. Le choix retenu est l'**anonymisation** plutôt que l'effacement : l'e-mail devient `deleted_<id>@deleted.invalid`, le pseudo `deleted_<8 caractères>`, et le mot de passe est remplacé par une valeur aléatoire. Les données personnelles disparaissent, mais les messages restent pour que les conversations des autres participants demeurent compréhensibles.

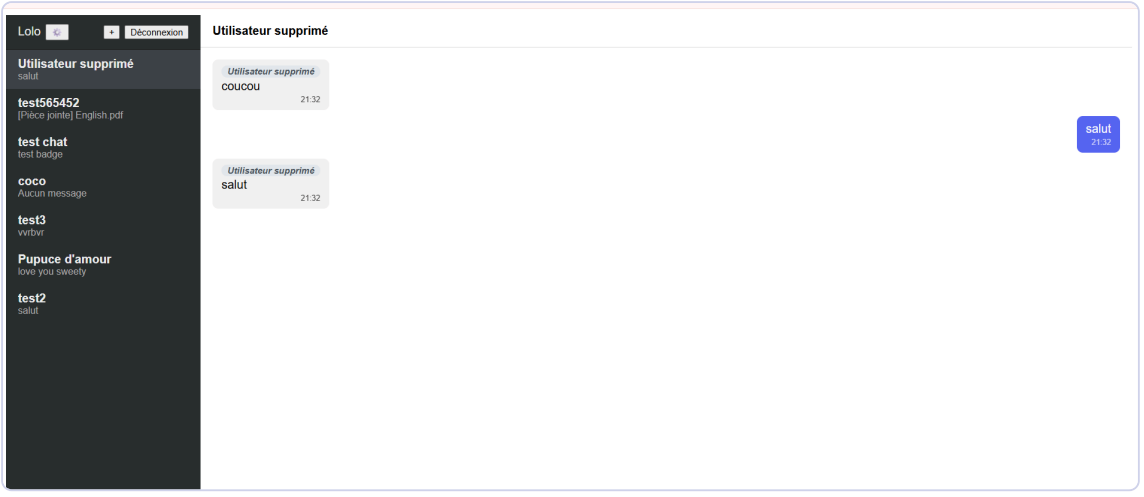


Profil



Confirmation de suppression

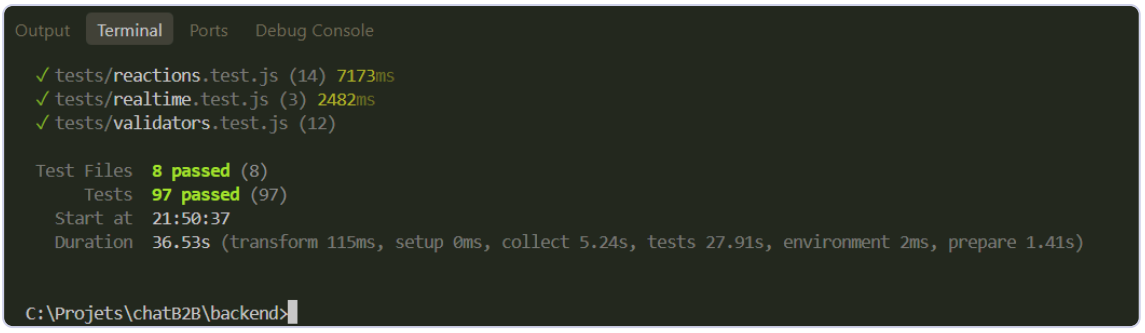
Un compte supprimé ne peut plus se connecter (middleware HTTP et Socket.io). Une conversation 1 à 1 avec un compte supprimé passe en **lecture seule** (réponse 403 à l'envoi) ; les groupes continuent. Les conversations qui n'ont plus aucun membre actif sont supprimées. L'interface affiche le badge « Utilisateur supprimé » (mise à jour au rechargement de la page).



Conversation avec un utilisateur supprimé

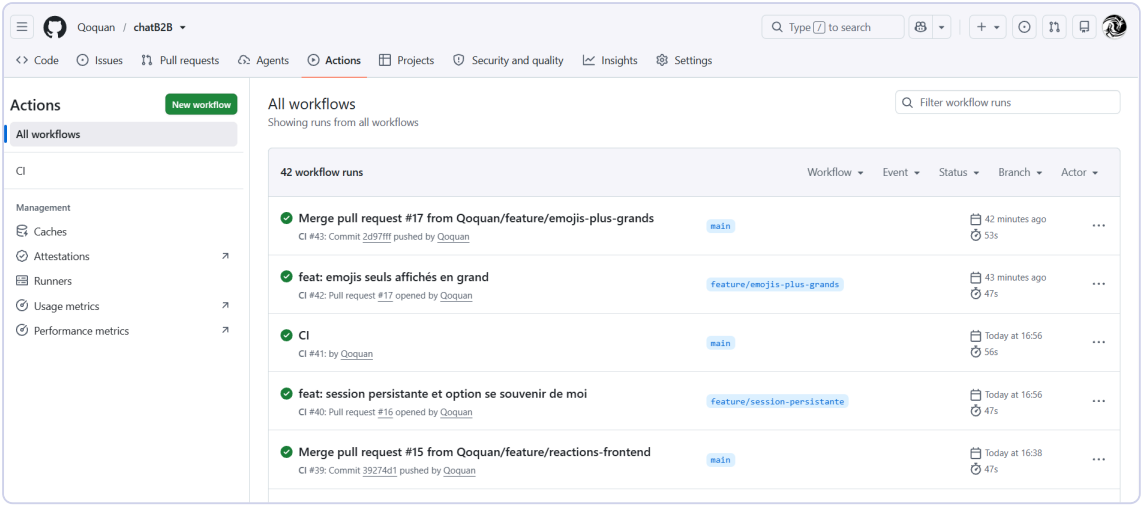
14. Qualité logicielle et CI/CD

Le backend est couvert par **97 tests automatisés** répartis en 8 fichiers (Vitest, supertest et socket.io-client), qui couvrent l'authentification, les validateurs, les conversations, les pièces jointes, les réactions, la suppression de compte et le temps réel. Le code est contrôlé par **ESLint** et formaté par **Prettier**.



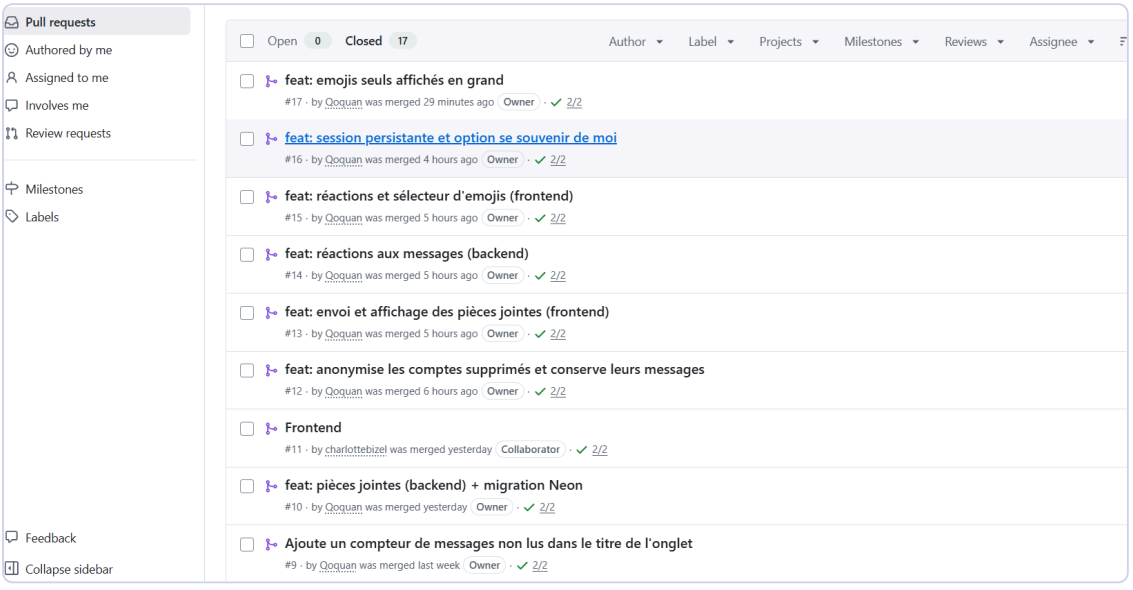
Exécution des tests : 97 réussis sur 97

Une chaîne d'**intégration continue GitHub Actions** s'exécute à chaque push et à chaque Pull Request vers `main`. Le job backend utilise un service PostgreSQL 16, lance lint, vérification du format, `prisma generate` + `db push`, puis les tests. Le job frontend lance lint et build.



Exécutions GitHub Actions toutes réussies

Le travail suit le **GitHub Flow** : une branche par fonctionnalité, une Pull Request relue par le collègue, CI verte obligatoire avant fusion.



Liste des Pull Requests

15. Sécurité

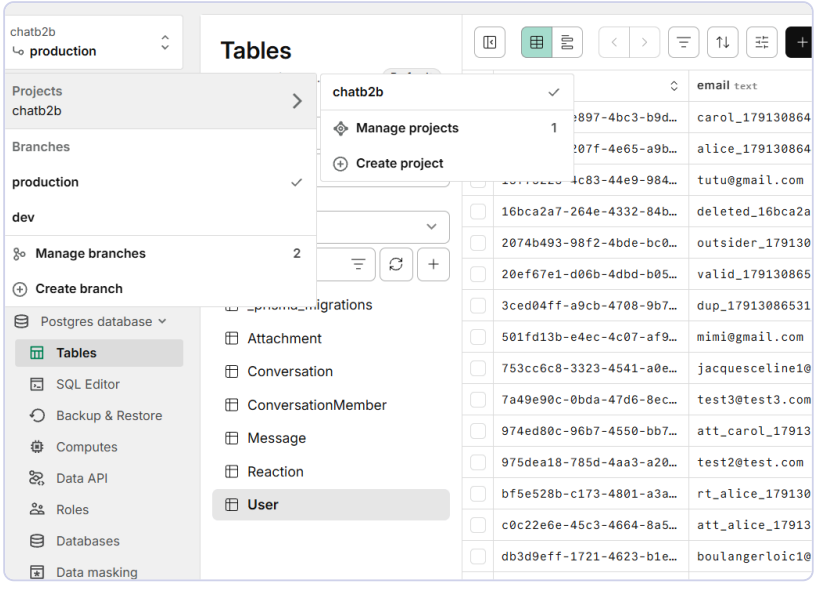
Mesures en place :

- Mots de passe hachés avec bcrypt, jamais stockés en clair ni renvoyés par l'API.
- Authentification par JWT signé, durée limitée (1 jour, ou 8 jours avec « se souvenir de moi ») ; routes API et connexions Socket.io protégées.
- Validation des entrées côté serveur (validateurs dédiés, testés) et requêtes Prisma paramétrées (pas d'injection SQL).
- Contrôle d'accès : seul un membre d'une conversation peut lire ses messages et télécharger ses fichiers.
- Fichiers : taille limitée, liste blanche de types, vérification des magic bytes, pas de SVG/HTML.
- Secrets (`JWT_SECRET`, URL de base de données) fournis par variables d'environnement, absents du dépôt.
- Comptes supprimés rejetés par les middlewares HTTP et Socket.io.

Limites assumées (voir chapitre 19) : pas d'en-têtes `helmet`, pas de limitation de débit, pas de jeton de rafraîchissement, et le JWT dans le stockage du navigateur reste exposé en cas de faille XSS.

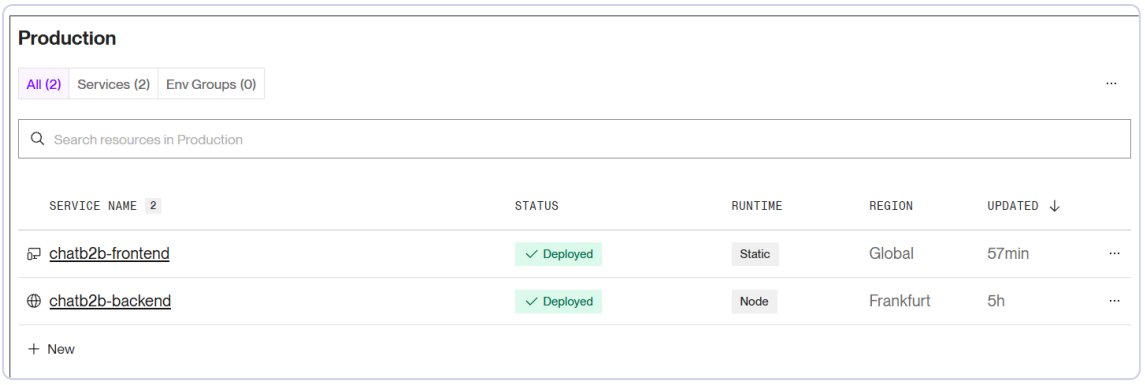
16. Déploiement et environnements

Deux environnements sont séparés sur Neon : la branche **production**, utilisée par le site en ligne, et la branche **dev**, utilisée en local par les développeurs (le fichier `.env` local pointe vers dev). Les migrations Prisma (`prisma migrate deploy`) sont appliquées automatiquement au déploiement sur Render. Cinq migrations existent : initialisation, avatar et créateur de groupe, pièces jointes, champ `deletedAt` des utilisateurs, réactions.



Base de données Neon et branches

Sur Render, les deux services du projet sont déployés : `chatb2b-frontend` (site statique) et `chatb2b-backend` (service Node.js, région Francfort).



Services Render : frontend et backend déployés

Extrait du journal d'un déploiement du backend (7 octobre 2026, après la fusion de la PR #17). On y voit la commande de build, la génération du client Prisma, la vérification des migrations, puis le démarrage du service :

```
==> Cloning from https://github.com/Qoquan/chatB2B
==> Running build command 'npm install && npx prisma generate && npx prisma migrate
deploy'...
✔ Generated Prisma Client (v5.22.0) to ./node_modules/@prisma/client
5 migrations found in prisma/migrations
No pending migrations to apply.
==> Build successful
==> Running 'npm start'
Serveur ChatB2B démarré sur le port 10000
==> Your service is live
```

Le message « No pending migrations to apply » signifie que la base est déjà à jour : lorsqu'une nouvelle migration est fusionnée, la même commande l'applique automatiquement avant le démarrage.

17. Sauvegarde et restauration

Le projet protège deux choses : le code et les données. Les copies sont conservées hors de la plateforme d'hébergement, et la restauration de la base a été testée le 28 octobre 2026.

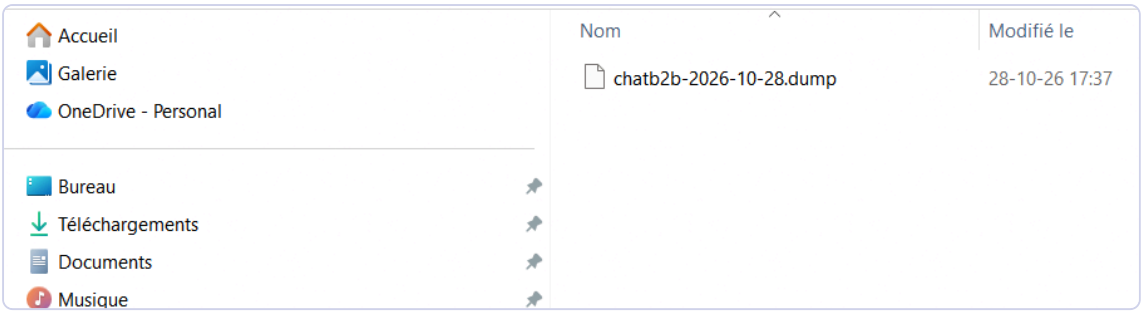
Élément	Où il se trouve	Sauvegarde
Code source	GitHub (`Qoquan/chatB2B`) et clone local de chaque membre	Copie miroir du dépôt et fichier « bundle » Git contenant tout l'historique
Base de données	Neon, branche `production`	Fichier `pg_dump` au format personnalisé
Pièces jointes	Colonne `BYTEA` de la base	Incluses dans le fichier `pg_dump`
Secrets (`JWT_SECRET`, `DATABASE_URL`, `DIRECT_URL`)	Variables d'environnement Render et fichier `.env` local	Copie privée, jamais dans le dépôt

17.1 Sauvegarde de la base de données

La sauvegarde utilise l'URL de connexion directe de Neon (sans pooler) et produit un fichier au format personnalisé de PostgreSQL. Le fichier obtenu le 28 octobre 2026 pèse 2,2 Mo.

```
pg_dump "<URL de connexion directe Neon>" -Fc -f chatb2b-2026-10-28.dump
```

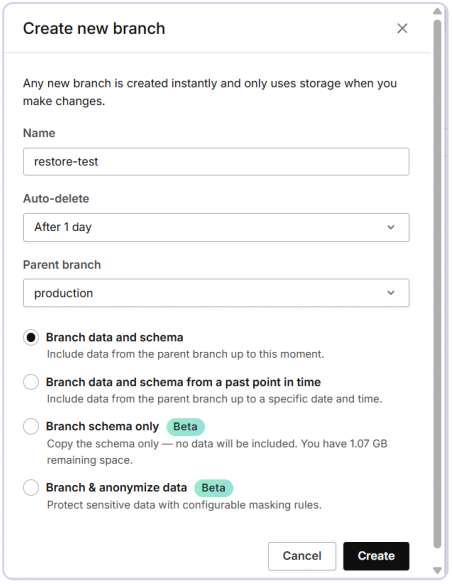
Deux règles s'appliquent : le mot de passe de la base n'est jamais écrit dans un fichier du dépôt, et le fichier de sauvegarde est rangé hors du dépôt, car il contient des adresses e-mail et des mots de passe hachés alors que le dépôt est public.



Fichier de sauvegarde chatb2b-2026-10-28.dump

17.2 Restauration et test

Pour ne jamais toucher à la production, la restauration a été testée sur une branche Neon jetable, `restore-test`. Elle est créée à partir de `production` avec le schéma seul, sans données, et supprimée automatiquement après un jour.



Création de la branche Neon restore-test

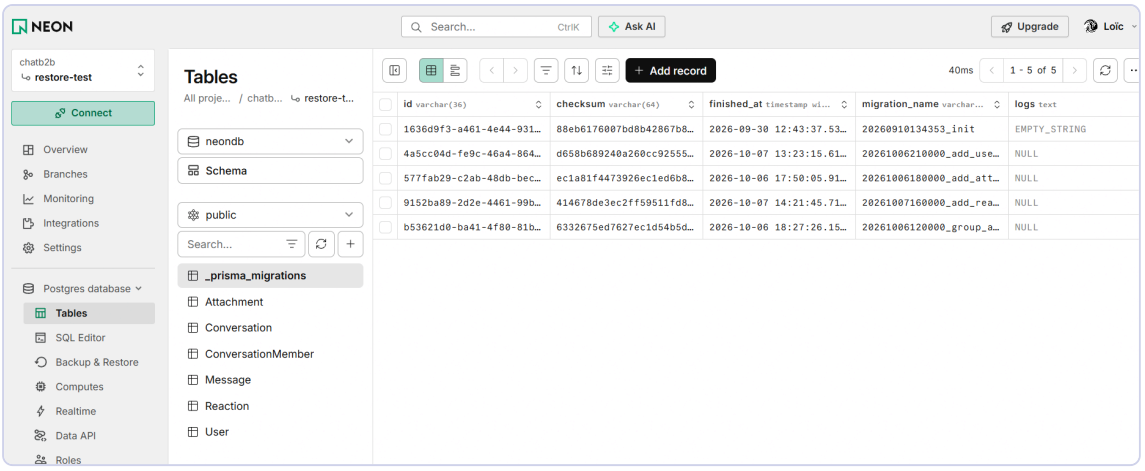
La commande suivante recrée les tables à partir du fichier de sauvegarde :

```
pg_restore --clean --if-exists --no-owner -d "<URL de la branche restore-test>" chatb2b-2026-10-28.dump
```

Résultat : les sept tables de l'application (`User`, `Conversation`, `ConversationMember`, `Message`, `Attachment`, `Reaction` et `_prisma_migrations`) existent dans la branche de test. Comme cette branche ne contenait aucune donnée au départ, la présence des cinq migrations dans `_prisma_migrations` montre que le fichier a bien rechargé les données.

Deux erreurs s'affichent pendant la restauration : Neon refuse deux commandes `ALTER DEFAULT PRIVILEGES` liées à ses droits internes (rôle `cloud_admin`). Elles n'ont aucun effet sur les données de

l'application, et `pg_restore` termine avec le message « errors ignored on restore: 2 ».



Tables restaurées sur la branche restore-test

17.3 Limites et suite

Les sauvegardes sont manuelles : elles sont faites avant un jalon important, comme le rendu ou une grosse mise en production. Une automatisation est possible avec une tâche planifiée GitHub Actions qui lance `pg_dump` chaque nuit. Le fichier devrait alors être envoyé vers un stockage privé, car les fichiers d'un dépôt public sont lisibles par tout le monde. Neon permet aussi de créer une branche instantanée avant un changement risqué.

18. Difficultés rencontrées et leçons

- **Incident de merge** : une fusion mal résolue a réintroduit un ancien état du code et cassé un déploiement. *Leçon* : toujours synchroniser sa branche avec `main` avant la PR, lire le diff, et laisser la CI bloquer les fusions.
- **Migrations sur Neon** : un conflit entre le schéma réel de la base et l'historique des migrations a demandé un travail minutieux (dossier vs fichier de migration, schéma divergent). *Leçon* : ne jamais modifier la base à la main sans migration correspondante, et tester d'abord sur la branche `dev`.
- **Écran blanc en production** : un caractère invalide a fait planter le rendu. *Leçon* : tester le build de production avant de fusionner.
- **Comptes de test en production** : des tests ont créé des comptes dans la base de production. Un script de nettoyage ciblé (adresses `@test.chatb2b.local`) a été préparé et testé en local ; il a été exécuté en production le 7 octobre 2026 : 13 comptes de test ont été supprimés, avec leurs messages, et 7 comptes ont été conservés. *Leçon* : séparer strictement les bases de test et de production.
- **Outils** : fichier renommé avec une casse différente (invisible sous Windows, bloquant sous Linux en CI), fichiers supprimés par erreur (récupérés avec un `git restore` ciblé). *Leçon* : la CI sous Linux révèle des erreurs que le poste local masque.

19. Limites connues et perspectives

Non réalisé ou limité : pas de pagination des messages (tout l'historique est chargé) ; pas de réinitialisation de mot de passe ni de vérification d'e-mail ; pas de tests frontend ; le badge « Utilisateur

supprimé » n'est pas mis à jour en temps réel ; la notification native du navigateur est variable selon les navigateurs.

Sécurité à renforcer : `helmet`, limitation de débit, jetons de rafraîchissement, stockage du jeton hors de `localStorage`, correction des 16 vulnérabilités signalées par `npm audit` (dépendances de développement antérieures au projet).

Sauvegardes : elles sont manuelles et non automatisées (voir chapitre 17). La restauration de la base a été testée sur une branche Neon jetable.

Perspectives : pagination infinie des messages, indicateurs de lecture, recherche, refonte graphique (CSS / UI / UX), documentation (README, `render.yaml`) à mettre à jour.

20. Conclusion

ChatB2B est une application de messagerie fonctionnelle et en production : messages en temps réel, groupes, pièces jointes sécurisées, réactions, notifications, session persistante et suppression de compte conforme aux principes du RGPD. Le projet repose sur une base de qualité solide (97 tests, CI, Pull Requests avec relecture avant fusion, déploiement automatisé).

Au-delà de la technique, il nous a appris à travailler à deux avec un vrai flux de collaboration, à gérer des environnements séparés, et à traiter les incidents plutôt que de les subir. Les limites listées au chapitre 19 sont identifiées et constituent la feuille de route naturelle de la suite.

21. Annexes

- Dépôt GitHub : https://github.com/Qoquan/chatB2B
- Membres : Loïc Boulanger (backend), Charlotte Bizel (frontend)
- Pull Requests fusionnées : voir la liste du chapitre 2 et la capture du chapitre 14.
- Lien du site déployé : https://chatb2b-frontend.onrender.com